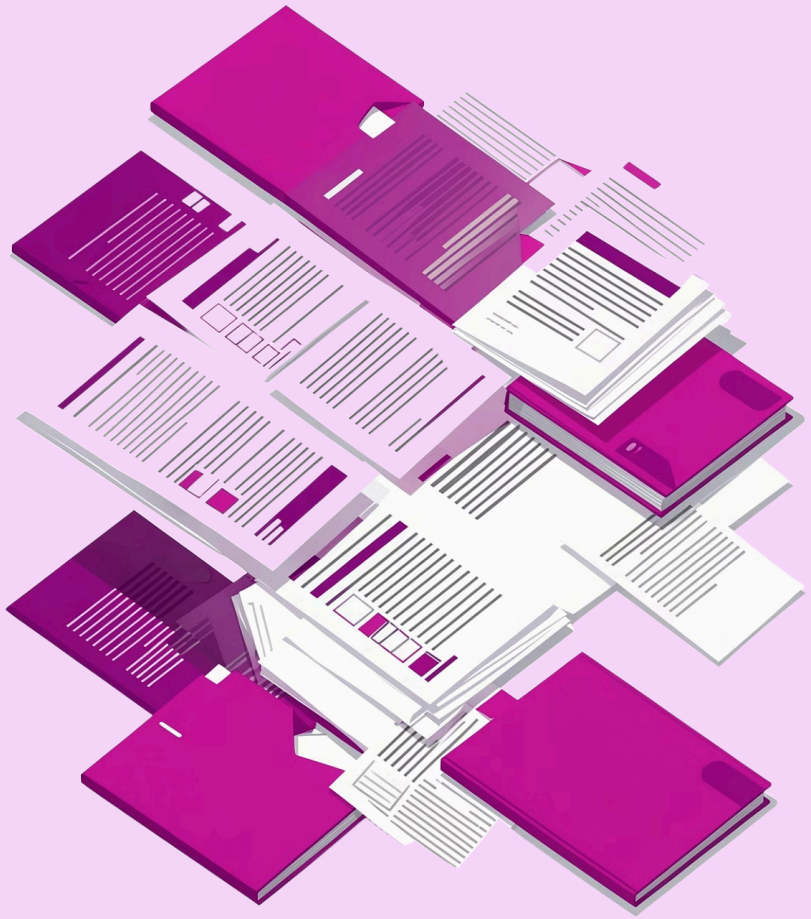
RAG Document Agent: Intelligent Document Q&A System

A technical deep-dive into the architecture and implementation of a Retrieval-Augmentation-Generation pipeline for enterprise document intelligence.



The Problem: Unstructured Document Data

Scattered Knowledge

Organizations struggle to extract insights from scattered PDFs, Word docs, and spreadsheets — data locked in silos with no unified access layer.

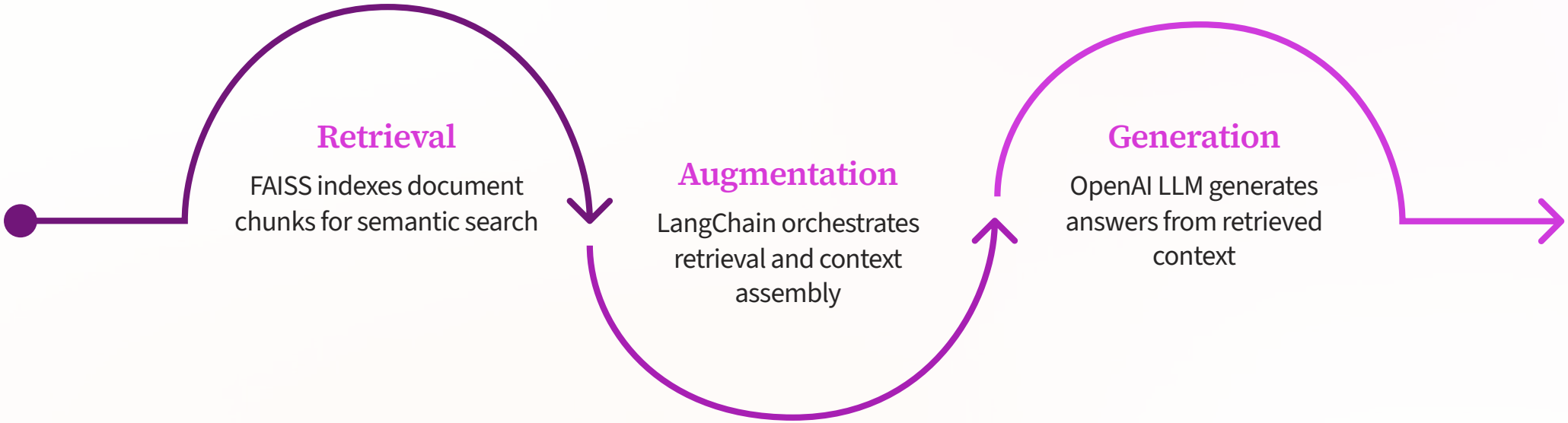
Manual Review Bottleneck

Manual document review is time-consuming and error-prone — engineers waste cycles searching instead of building.

Accuracy Imperative

There is a critical need for instant, accurate answers grounded strictly in actual document content — not hallucinated guesses.

Solution Architecture: RAG Pipeline



Each stage is independently configurable, allowing developers to tune performance, model selection, and retrieval depth to match their use case.

Multi-Format Document Processing

1 Dynamic Loader Selection

Handles PDF, Word (.docx/.doc), Excel (.xlsx/.xls), and plain text through a unified dispatch layer.

2 Unified Extraction Pipeline

Normalizes all formats into searchable, consistently sized chunks — regardless of source format or structure.

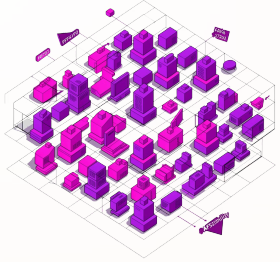
3 Metadata Preservation

Preserves document structure and metadata for accurate, page-level citations in every generated answer.

Vector Embeddings & Retrieval

Semantic Embedding

OpenAI `text-embedding-3-small` converts document chunks into high-dimensional semantic vectors that capture meaning, not just keywords.



FAISS Indexing

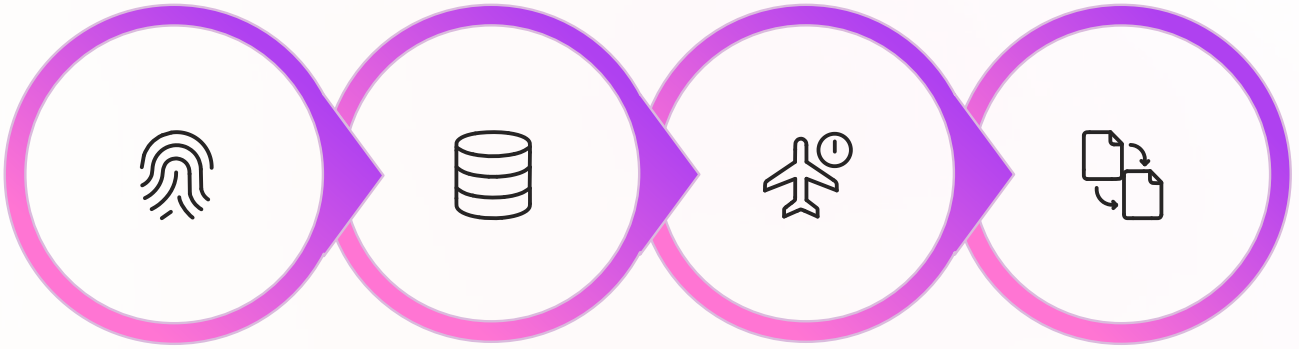
FAISS indexes vectors for fast approximate nearest-neighbor similarity search, scaling efficiently across large document corpora.

Configurable Top-K

Configurable Top-K retrieval returns the most semantically relevant chunks for each query — giving the LLM precisely the right context.

Smart Caching & Performance

Redundant embedding and re-indexing are the silent killers of RAG system performance. The agent eliminates them entirely through a layered caching strategy.



File Signature Detection

`file_sig` hash detects document changes and invalidates cache only when the source file actually changes.

Session-State Index Reuse

The FAISS index is persisted in session state and reused across all queries on the same document — zero re-indexing cost.

Zero Redundant Embeddings

Eliminates duplicate API calls to the embedding model, reducing latency and cutting operational cost on large documents.

Configurable RAG Parameters

Developers retain full control over every dimension of retrieval and generation performance.

Chunk Size & Overlap

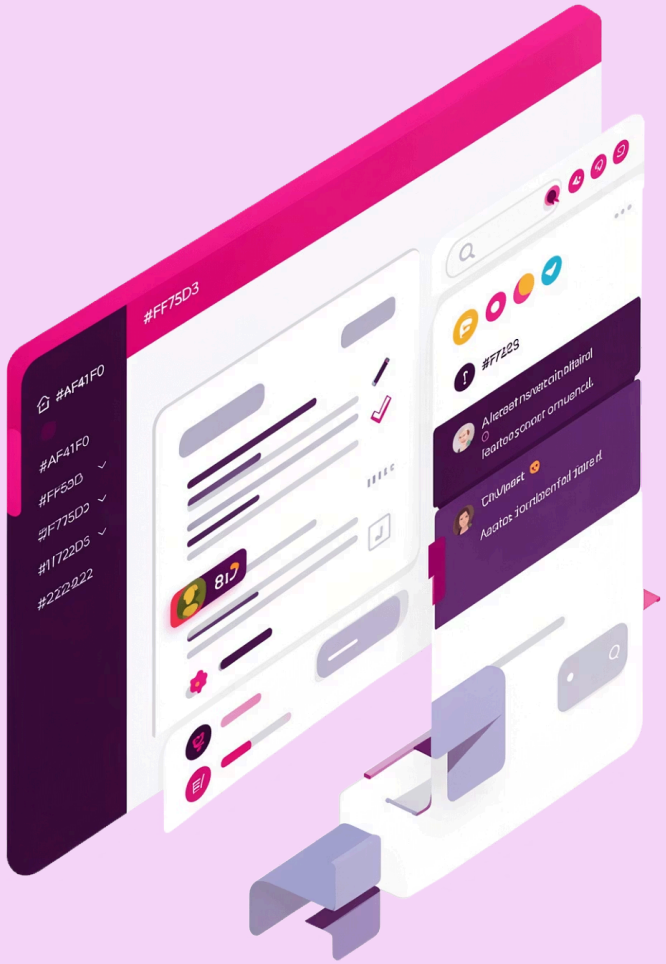
Fine-tune the context window size and chunk overlap to balance retrieval precision against context richness. Larger overlaps reduce boundary artifacts; smaller chunks improve specificity.

Model Selection

Choose between `gpt-4o-mini` for low-latency, high-throughput scenarios or `gpt-4o` for complex multi-step reasoning over dense technical documents.

Top-K Setting

Control the exact number of retrieved chunks injected into the LLM context per query — tuning the precision-recall tradeoff for your document type and query complexity.



Key Features & Transparency

Ground-Truth Enforcement

Strict system prompts constrain the LLM to respond **only** from retrieved document context — hallucination is architecturally prevented, not just discouraged.

Built-In Chunk Viewer

Every answer is accompanied by a built-in chunk viewer with page-level citations, giving developers and end-users full source transparency and auditability.

Streamlit UI

A clean, responsive Streamlit interface optimized for seamless document upload, query interaction, and result exploration — ready to deploy without frontend overhead.